

UNSUPERVISED MACHINE LEARNING

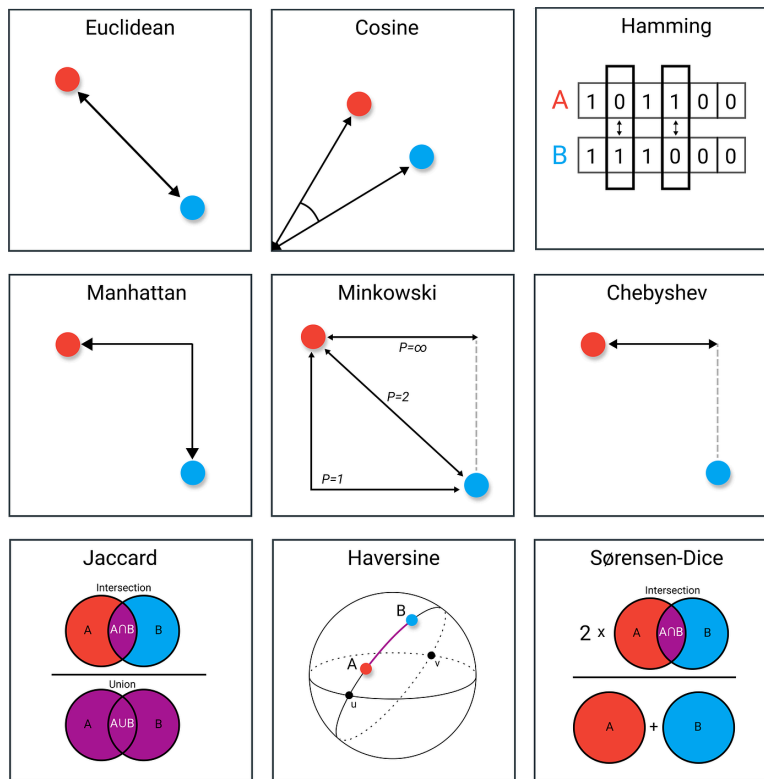
MODULE 2:

DISTANCE METRICS & COMPUTATIONAL HURDLES

TABLE OF CONTENTS

Introduction to Distance Metrics.....	2
Euclidean Distance (L2 Metric).....	3
Manhattan Distance (L1 or City Block Distance).....	4
Cosine Distance.....	5
Jaccard Distance.....	6
Key Takeaways.....	7

Introduction to Distance Metrics



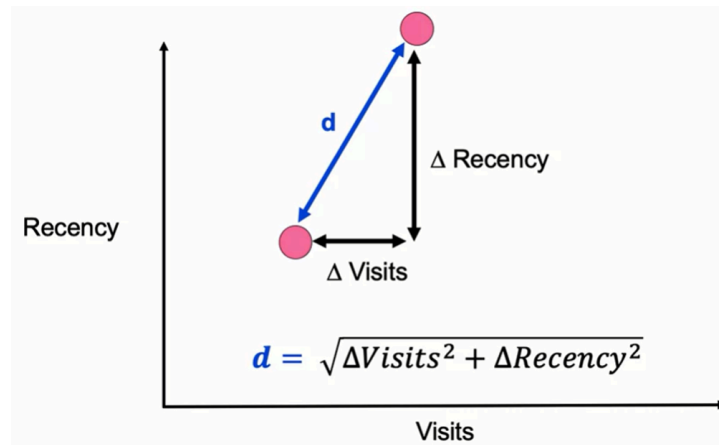
Main ideas:

- Clustering algorithms depend heavily on how **distance (or similarity)** is defined.
- The **choice of distance metric** greatly impacts clustering performance and results.
- Different metrics suit different data types; often the best choice is found **empirically** through testing.

Most common distance metrics covered:

- Euclidean (L2)
- Manhattan (L1 or City Block)
- Cosine
- Jaccard

Euclidean Distance (L2 Metric)



Concept:

- The **most intuitive** and commonly used metric, including in **K-Means**.
- Measures straight-line distance between two points in space.

Formula:

$$d(A, B) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

In n dimension:

$$d(A, B) = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$$

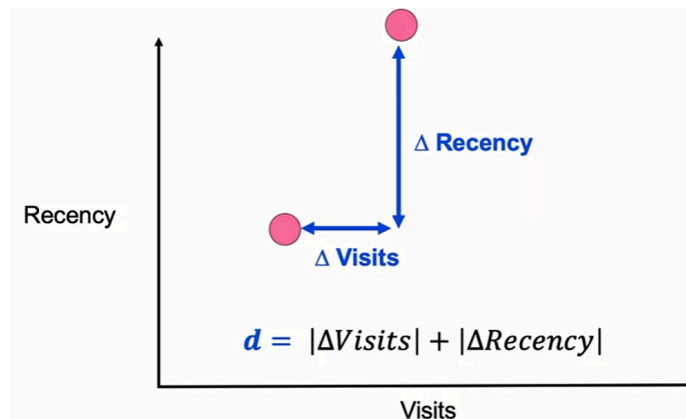
```
import numpy as np

a = np.array([2, 3])
b = np.array([5, 7])
distance = np.linalg.norm(a - b) # Euclidean distance
```

Notes:

- Sensitive to **scale and outliers**.
- Performance decreases in **high-dimensional** data (curse of dimensionality).

Manhattan Distance (L1 or City Block Distance)



Concept:

- Measures distance by summing **absolute differences** along each axis.
- Represents movement along grid-like paths (like Manhattan streets).

Formula:

$$d(A, B) = \sum_{i=1}^n |a_i - b_i|$$

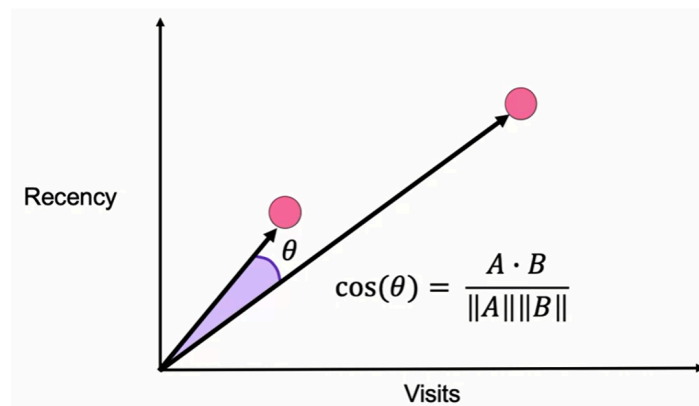
```
import numpy as np

a = np.array([2, 3])
b = np.array([5, 7])
manhattan = np.sum(np.abs(a - b))
```

Notes:

- Always $\geq$ Euclidean distance (except when points align perfectly).
- Performs better than Euclidean in high-dimensional spaces because it separates distances more clearly.

Cosine Distance



Concept:

- Focuses on the **angle** between two vectors, not their magnitude.
- Measures **directional similarity**, making it suitable for text and document data.

Formula:

$$\text{Cosine Similarity: } \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

$$\text{Cosine Distance: } d_{\text{cosine}}(A, B) = 1 - \frac{A \cdot B}{\|A\| \|B\|}$$

```

from numpy import dot
from numpy.linalg import norm

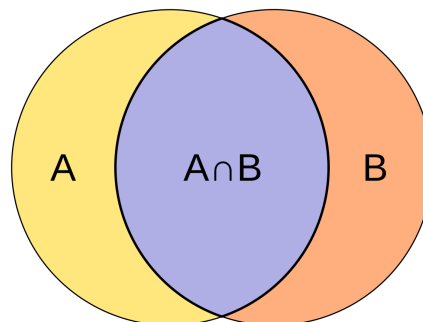
a = np.array([3, 10])
b = np.array([30, 100])
cosine_distance = 1 - (dot(a, b) / (norm(a) * norm(b)))

```

Notes:

- Insensitive to **magnitude scaling** — if two vectors lie on the same ray from the origin, distance = 0.
- Robust against the **curse of dimensionality**.
- Common in **text mining**, **document similarity**, or **word embeddings**.

Jaccard Distance



Concept:

- Measures dissimilarity between **sets** (commonly used for text).
- Based on **intersection over union (IoU)** of unique elements.

Formula:

$$d_{\text{Jaccard}}(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|}$$

Example:

- Sentence A: “I like chocolate ice cream”
→ {I, like, chocolate, ice, cream}
- Sentence B: “Do I want chocolate cream or vanilla cream”
→ {Do, I, want, chocolate, cream, or, vanilla}

$$|A \cap B| = 3, \quad |A \cup B| = 9$$

$$d_{\text{Jaccard}}(A, B) = 1 - \frac{3}{9} = \frac{2}{3} = 0.67$$

```
A = set("I like chocolate ice cream".split())
B = set("Do I want chocolate cream or vanilla cream".split())

jaccard_distance = 1 - len(A & B) / len(A | B)
```

Use cases:

- **Text clustering and topic grouping**
- **Document similarity** for token sets

Key Takeaways

Metric	Main Use	Strength	Limitation
Euclidean (L2)	Coordinate-based data	Intuitive, simple	Sensitive to scaling & high dimensions
Manhattan (L1)	High-dimensional numeric data	Distinguishes distances well	Still scale-sensitive
Cosine	Text/document embeddings	Ignores magnitude, angle-based	Not useful for zero-centered data
Jaccard	Sets/text data	Measures overlap directly	Ignores order and frequency